

PPS - Scala Stream Fusion

Leonardo Naddei

8 agosto 2026

Indice

1	Metaprogrammazione e generazione statica del codice in Scala 3	3
1.1	Multi-stage programming in Scala 3	3
1.2	Metaprogrammazione generativa e analitica	4
1.3	Dalle espressioni all'AST tipizzato	4
1.4	Tipi staged e tipi riflessivi	4
1.5	Beta reduction delle funzioni	5
1.6	Simboli, variabili generate e hygiene	6
1.7	Requirement specification	6
1.7.1	Requisiti di business	6
1.7.2	Modello di dominio	6
1.7.3	Requisiti funzionali utente	6
1.7.4	Requisiti funzionali di sistema	6
1.7.5	Requisiti non funzionali	6
1.7.6	Requisiti di implementazione	6
1.8	Design architetturale	6
1.9	Design di dettaglio	6
1.9.1	Operazioni Supportate	6
1.9.2	FlatMap (Espansione Monadica e Appiattimento)	8
1.10	Implementazione	9
1.11	Testing	9
1.12	Retrospettiva	9

Abstract

Il presente lavoro descrive la realizzazione di una libreria per Scala 3 che consenta di definire pipeline di elaborazione secondo un paradigma dichiarativo, ottenendo al contempo prestazioni comparabili a quelle di codice imperativo specializzato manualmente.

Basandosi sul concetto di stream fusion, introdotto in Stream Fusion: From Lists to Streams to Nothing at All, e sfruttando le funzionalità di metaprogrammazione offerte dalle macro di Scala 3, è stata sviluppata una libreria capace di trasformare pipeline dichiarative in codice specializzato ed efficiente generato a compile-time. La libreria offre inoltre un'architettura estendibile per la definizione di nuovi operatori terminali, permettendo di integrare nuovi comportamenti nella fase di fusione senza rinunciare alle ottimizzazioni statiche.

Tale generazione statica permette di eliminare i principali overhead tipici delle astrazioni funzionali su JVM. L'analisi del bytecode prodotto conferma la completa fusione delle operazioni sequenziali in un unico ciclo imperativo e l'inlining diretto delle lambda nel callsite. Ciò azzerava l'uso di istruzioni `invokedynamic`, elimina il dispatch indiretto (evitando call site polimorfici o megamorfici) e rimuove le allocazioni di collezioni intermedie, iteratori e closure, azzerando al contempo il boxing dei tipi primitivi.

Per certi aspetti, il modello ottenuto richiama il paradigma di genericità dei template C++: il codice viene specializzato in fase di compilazione rispetto ai tipi e alla specifica composizione delle operazioni, quando tali informazioni sono disponibili staticamente. Le macro di Scala aggiungono tuttavia una componente esplicita di metaprogrammazione, permettendo non soltanto di istanziare codice generico, ma anche di analizzare, trasformare e ottimizzare la struttura della pipeline prima della sua emissione.

I benchmark condotti tramite JMH mostrano uno speedup significativo e un consumo di memoria drasticamente ridotto rispetto alle librerie standard. Il risultato è un framework che preserva l'espressività, la composizionalità e la type safety tipiche della programmazione funzionale.

Capitolo 1

Metaprogrammazione e generazione statica del codice in Scala 3

Il sistema di metaprogrammazione di Scala 3 segue un approccio di *multi-stage programming*: una parte del programma viene eseguita durante uno stage precedente e produce codice destinato a uno stage successivo. Questo modello permette di costruire macro che generano codice ben tipato e safe per costruzione [?, ?].

Nel progetto sviluppato, la metaprogrammazione viene utilizzata in entrambe le sue forme principali: analitica, per ispezionare e destrutturare la pipeline scritta dall'utente, e *generativa*, per produrre il codice che ne realizza l'esecuzione.

1.1 Multi-stage programming in Scala 3

Le due primitive fondamentali del sistema di metaprogrammazione staged di Scala 3 sono le *quote* e gli *splice*.

Una quote

```
'{ expression }
```

costruisce una rappresentazione tipizzata del codice contenuto al suo interno. Se **expression** produce un valore di tipo **T**, il risultato della quote ha tipo

`Expr[T]`

e rappresenta codice che produrrà un valore di tipo **T** nello stage successivo.

Uno splice:

```
${ expression }
```

opera invece nella direzione opposta e inserisce all'interno del codice quotato il frammento di programma rappresentato da un'espressione `Expr[T]`.

Il classico esempio di generazione staged è l'elevamento a potenza con esponente noto durante la compilazione:

```
def powCode(x: Expr[Int], n: Int)(using Quotes): Expr[Int] =  
  if n == 0 then  
    Expr(1)  
  else  
    '{ $x * ${ powCode(x, n - 1) } }
```

La ricorsione di `powCode` viene eseguita durante l'espansione della macro. Per un esponente pari a tre, il frammento generato è concettualmente equivalente a

```
x * x * x * 1
```

e non contiene quindi né il ciclo né la ricorsione impiegata dal metaprogramma per costruirlo.

1.2 Metaprogrammazione generativa e analitica

Una macro puramente generativa costruisce un nuovo frammento di programma a partire da informazioni note durante la compilazione. Nel progetto è tuttavia necessario anche compiere l'operazione inversa: ricevere un frammento di programma, riconoscerne la struttura e decomporlo nei suoi componenti.

Scala 3 supporta questa forma di metaprogrammazione attraverso il pattern matching sul codice quotato. Ad esempio, una generica espressione può essere analizzata con un pattern della forma

```
expr match {  
  case '{ someFunction($argument) } =>  
    ...  
}
```

Per i casi più semplici sarebbe possibile eseguire questa analisi utilizzando esclusivamente quoted pattern. Nel progetto, tuttavia, è necessario accedere a dettagli più fini dell'albero generato dal compilatore, rendendo necessario il passaggio alla Reflection API.

1.3 Dalle espressioni all'AST tipizzato

Il tipo `Expr[T]` offre un'interfaccia ad alto livello sul codice. Tale rappresentazione nasconde parte della struttura interna dell'espressione. Quando è necessario analizzare direttamente il programma è possibile convertire una `Expr[T]` nel relativo nodo dell'AST prodotto dal compilatore:

```
expr.asTerm
```

La Reflection API espone quindi la struttura tipizzata prodotta dal compilatore. Tra i nodi più importanti utilizzati nel parser vi sono:

- `Apply`, per l'applicazione di una funzione;
- `TypeApply`, per un'applicazione contenente parametri di tipo;
- `Select`, per la selezione di un membro;
- `Ident`, per il riferimento a un identificatore;
- `Block`, per un blocco contenente dichiarazioni e un'espressione finale;
- `Inlined`, per rappresentare il risultato di un'espansione inline;
- `ValDef`, `Assign` e `Ref`, utilizzati durante la generazione di nuovo codice.

Questo livello di rappresentazione è sostanzialmente più espressivo rispetto alle sole quote.

1.4 Tipi staged e tipi riflessivi

Scala 3 mette a disposizione due rappresentazioni complementari:

`Type[T]` rappresenta il tipo `T` nell'API staged ed è utilizzabile insieme a `Expr[T]`, quote e splice;

`TypeRepr` rappresenta invece un tipo all'interno della Reflection API e permette di ispezionarne direttamente la struttura.

Data una `TypeRepr`, il metodo `.asType` restituisce un valore `Type[?]` sul quale è possibile eseguire pattern matching:

```
item.asType match {
  case '[elem] => ...
}
```

Un esempio è l'estrazione del tipo per la creazione di un iteratore da un singolo elemento:

```
private def createSourceOf(item: Term): ParsedTree = {
  item.asType match {
    case '[elem] =>
      val singleExpr = item.asExprOf[elem]
      val iterableExpr: Expr[Iterable[elem]] =
        '{ Iterable.single[elem]($ { singleExpr }) }
      ...
  }
}
```

Questo esempio contiene in pochi passaggi l'intero collegamento tra reflection e staged programming:

1. viene determinato il tipo del termine dell'AST ricevuto;
2. il tipo viene estratto in `elem` mediante il pattern `'[elem]`;
3. il `Term` originale viene convertito in `Expr[elem]`;
4. viene generato un nuovo frammento `Expr[Iterable[elem]]`;

1.5 Beta reduction delle funzioni

Un altro elemento importante è l'utilizzo della β -reduction, considerando una funzione:

$$f = x \mapsto e, \quad (1.1)$$

l'applicazione

$$f(a) \quad (1.2)$$

può essere riscritta sostituendo direttamente x con a all'interno di e .

Scala 3 rende disponibile questa trasformazione attraverso: `Expr.betaReduce(...)`, viene utilizzata per l'applicazione esplicita delle lambda eliminando invocazioni tramite `invokedynamic`. Prendendo ad esempio l'operazione `filter`:

```
val cond = Expr.betaReduce('{ $ { filter.predicate }($elem) })
```

Se la funzione ricevuta è una lambda direttamente disponibile alla macro, il suo corpo può quindi essere incorporato nel codice generato.

Una pipeline come

```
stream
  .filter(x => x > 0)
  .map(x => x * 2)
```

può pertanto contribuire a generare un corpo concettualmente equivalente a

```
if (elem > 0) {
  emit(elem * 2)
}
```

Invece che:

```
if (filter.test(elem)) {  
    emit(map.apply(elem))  
}
```

Nell'ultimo caso, `filter` e `map` denotano istanze materializzate dalla JVM tramite call site `invokedynamic`, la loro applicazione avviene attraverso una chiamata indiretta all'interfaccia funzionale, ad esempio `Function1.apply`. Questo introduce un call site il cui target deve essere risolto e, qualora diventi polimorfo o megamorfo, può ridurre l'efficacia di devirtualizzazione, inlining e ulteriori ottimizzazioni interprocedurali del JIT. Incorporando staticamente il corpo della lambda nell'IR generato, la macro elimina sia la rappresentazione della funzione come entità invocabile sia il relativo dispatch.

1.6 Simboli, variabili generate e hygiene

Nell'AST di Scala, un riferimento a una variabile è associato semanticamente al relativo `Symbol`. Il nome è solamente una delle proprietà del simbolo e non ne rappresenta l'identità. La Reflection API permette di creare nuove variabili attraverso: `Symbol.newVal(...)`, e di fare successivamente riferimento allo stesso simbolo attraverso `Ref(symbol)`. La definizione viene creata nell'AST tramite `ValDef(symbol, Some(initializer))`

La distinzione tra nome e simbolo garantisce inoltre l'*hygiene* della macro: variabili generate dalla macro non possono andare in conflitto con variabili omonime definite nel codice utente.

TODO: esempio creazione e uso

1.7 Requirement specification

1.7.1 Requisiti di business

1.7.2 Modello di dominio

1.7.3 Requisiti funzionali utente

1.7.4 Requisiti funzionali di sistema

1.7.5 Requisiti non funzionali

1.7.6 Requisiti di implementazione

1.8 Design architetturale

L'applicativo è composto da due aree: l'API utilizzabile e un compilatore che si occupa del parsing, ottimizzazione e creazione di codice ad hoc a partire dal codice delle macro

1.9 Design di dettaglio

design dell'API

- alberi di parsing

- estensione per enrichment

1.9.1 Operazioni Supportate

Dato un generico stream $\mathcal{S}(A)$ ovvero una sequenza (potenzialmente infinita) di elementi omogenei presi dal dominio del tipo A , ovvero $\mathcal{S}(A) \in A^* \cup A^\infty$.

Di seguito vengono formalizzate, in termini di firma funzionale, semantica algebrica e cardinalità di emissione, le operazioni di trasformazione intermedie supportate.

Map (Trasformazione Funzionale Element-wise)

L'operatore `map` rappresenta l'applicazione di una funzione di trasformazione deterministica priva di side effects su ogni singolo elemento del flusso.

Definizione 1.9.1 (Map). Sia $f : A \rightarrow B$ una funzione pura. L'operatore $\text{map}_f : \mathcal{S}(A) \rightarrow \mathcal{S}(B)$ trasforma la sequenza $\langle a_1, a_2, \dots \rangle$ nella sequenza $\langle f(a_1), f(a_2), \dots \rangle$.

- **Firma e Tipo:** $\text{map} : (A \rightarrow B) \rightarrow (\mathcal{S}(A) \rightarrow \mathcal{S}(B))$
- **Mappatura di Cardinalità:** $1 : 1$. Per ogni elemento in ingresso $a_i \in A$, l'operatore produce esattamente un elemento in uscita $b_i = f(a_i) \in B$.
- **Proprietà Algebriche:** Operazione *stateless* (priva di stato), locale e conservativa rispetto alla lunghezza del flusso ($|\mathcal{S}(B)| = |\mathcal{S}(A)|$).

Filter (Proiezione Predicativa)

L'operatore `filter` seleziona un sottoinsieme degli elementi del flusso originale sulla base del soddisfacimento di una condizione booleana.

Definizione 1.9.2 (Filter). Sia $p : A \rightarrow \mathbb{B}$ un predicato, dove $\mathbb{B} = \{\text{true}, \text{false}\}$. L'operatore $\text{filter}_p : \mathcal{S}(A) \rightarrow \mathcal{S}(A)$ preserva l'ordine degli elementi ed include l'elemento a_i nel flusso di uscita se e solo se $p(a_i) = \text{true}$.

- **Firma e Tipo:** $\text{filter} : (A \rightarrow \mathbb{B}) \rightarrow (\mathcal{S}(A) \rightarrow \mathcal{S}(A))$
- **Mappatura di Cardinalità:** $1 : \{0, 1\}$. Ogni elemento a_i genera 1 elemento se $p(a_i) = \text{true}$, e 0 elementi (scarto) se $p(a_i) = \text{false}$.
- **Proprietà Algebriche:** Operazione *stateless*, locale, con conservazione dell'ordine relativo ($|\mathcal{S}(A)_{\text{out}}| \leq |\mathcal{S}(A)_{\text{in}}|$).

Limit (Troncamento del Flusso)

L'operatore `limit` definisce una delimitazione posizionale sulla dimensione massima del flusso processato.¹

Definizione 1.9.3 (Limit). Sia $n \in \mathbb{N}_0$ un intero non negativo indicante la cardinalità massima del flusso desiderato. L'operatore $\text{limit}_n : \mathcal{S}(A) \rightarrow \mathcal{S}(A)$ estrae il prefisso di lunghezza al più n del flusso $\mathcal{S}(A)$, interrompendo la propagazione degli elementi successivi.

- **Firma e Tipo:** $\text{limit} : \mathbb{N}_0 \rightarrow (\mathcal{S}(A) \rightarrow \mathcal{S}(A))$
- **Mappatura di Cardinalità:** $1 : \{0, 1\}$ (dipendente dall'indice posizionale i). Per l'indice $i \leq n$ la cardinalità è $1 : 1$, mentre per $i > n$ la cardinalità trasla a $1 : 0$.
- **Proprietà Algebriche:** Operazione *stateful* (richiede il mantenimento di un contatore interno dei messaggi) e con valutazione a corto circuito (*short-circuiting*).

¹Se il troncamento è guidato da un predicato $p : A \rightarrow \mathbb{B}$ anziché da una dimensione fissa n , l'operatore assume la semantica di `takeWhile`.

Skip (Scarto Posizionale Iniziale)

L'operatore `skip` scarta i primi n elementi dal flusso di input, lasciando passare inalterata la restante porzione della sequenza.²

Definizione 1.9.4 (Skip). Sia $n \in \mathbb{N}_0$ un offset intero. L'operatore $\text{skip}_n : \mathcal{S}(A) \rightarrow \mathcal{S}(A)$ sopprime i primi n elementi del flusso $\mathcal{S}(A)$ e propaga integralmente la sequenza a partire dall'elemento $(n + 1)$ -esimo.

- **Firma e Tipo:** $\text{skip} : \mathbb{N}_0 \rightarrow (\mathcal{S}(A) \rightarrow \mathcal{S}(A))$
- **Mappatura di Cardinalità:** $1 : \{0, 1\}$ (dipendente dall'indice posizionale i). Per l'indice $i \leq n$ la cardinalità è $1 : 0$, mentre per $i > n$ diventa $1 : 1$.
- **Proprietà Algebriche:** Operazione *stateful* basata su un contatore transitorio.

1.9.2 FlatMap (Espansione Monadica e Appiattimento)

L'operatore `flatMap` costituisce l'operatore di *bind* monadico per la struttura dati stream, combinando una trasformazione $1 \rightarrow N$ con il successivo appiattimento (*flattening*) delle sottosequenze generate.

Definizione 1.9.5 (FlatMap). Sia $g : A \rightarrow \mathcal{S}(B)$ una funzione che associa ad ogni elemento di tipo A un sotto-flusso di elementi di tipo B . L'operatore $\text{flatMap}_g : \mathcal{S}(A) \rightarrow \mathcal{S}(B)$ applica g ad ogni elemento $a_i \in \mathcal{S}(A)$ e concatena sequenzialmente i flussi risultanti:

$$\text{flatMap}_g(\langle a_1, a_2, \dots \rangle) = g(a_1) \parallel g(a_2) \parallel \dots$$

dove $\parallel$ rappresenta l'operatore di concatenazione ordinata tra flussi.

- **Firma e Tipo:** $\text{flatMap} : (A \rightarrow \mathcal{S}(B)) \rightarrow (\mathcal{S}(A) \rightarrow \mathcal{S}(B))$
- **Mappatura di Cardinalità:** $1 : N$, con $N \in \mathbb{N}_0 \cup \{\infty\}$. Ciascun elemento $a_i \in A$ genera un sotto-flusso $g(a_i)$ contenente da 0 a N elementi di tipo B .
- **Proprietà Algebriche:** Corrisponde all'operatore *Monadic Bind* ($\gg=$). Consente la variazione dinamica della struttura e della lunghezza complessiva del flusso.

Sintesi delle Operazioni e Proprietà Algebriche

In Tabella 1.1 viene riportato un quadro riassuntivo delle firme formali e delle caratteristiche strutturali delle operazioni analizzate.

Tabella 1.1: Confronto formale delle operazioni trasformative su flussi di dati.

Operazione	Firma di Dominio	Tipo di Trasformazione	Cardinalità ($1 : M$)	Gestione Stato
Map	$A \rightarrow B$	$\mathcal{S}(A) \rightarrow \mathcal{S}(B)$	$1 : 1$	Stateless
Filter	$A \rightarrow \mathbb{B}$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateless
Limit	$\mathbb{N}_0$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateful
Skip	$\mathbb{N}_0$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateful
FlatMap	$A \rightarrow \mathcal{S}(B)$	$\mathcal{S}(A) \rightarrow \mathcal{S}(B)$	$1 : N$ ($N \geq 0$)	Stateless / Contextual

²Nel caso in cui lo scarto sia basato su una condizione $p : A \rightarrow \mathbb{B}$, l'operatore assume la semantica di `dropWhile`.

1.10 Implmentazione

algoritmo implementato (push stream)

ottimizzazioni applicate ai parametri

enrichement: bubbling delle dichiarazioni

generazione di codice

implementazione del filter

implementazione del flatmapping

es semplice:

IncIter(from: Int): infinite lazy iterator, each next returns the next

FusedStream .from(List(1,2,3,4,5)) .skip(3) .flatMap(n -> Stream.of(IncIter(n*10)).limit(3))

.limit(10) .toList()

// Must be converted to

i = 0 j = 0

for i in 1: if(i++ >= 3) «< must inject condition on j too k = 0 iter = IncIter(i* 10) for

ii in iter if(k++ >= 3) break ——— This get injectd into the flatMap ——— if(j ++ >= 10)

break all push(ii) ———

esempio estensione con collector utilizzando i trait

1.11 Testing

1.12 Retrospettiva